

# Module 2: Object-Oriented Programming, Files, and NumPy

## Comprehensive Reference Document

EC Python for AI Students

---

*A thorough exposition of OOP, file I/O, exception management,  
and numerical computing with NumPy. Study alongside the companion notebook.*

---

## 1 Object-Oriented Programming Fundamentals

Object-oriented programming (OOP) organises software around *objects*—self-contained entities that bundle state (data, stored as **attributes**) with behaviour (operations, defined as **methods**). The central distinction is between a **class**, which serves as a blueprint or template, and an **instance** (object), which is a concrete realisation occupying memory. One class can produce arbitrarily many independent instances.

### 1.1 Defining Classes: `__init__`, `self`, Attributes, and Methods

A class is introduced with `class` followed by a PascalCase name. The special method `__init__` is the **constructor**, invoked automatically on instantiation. Every method must have `self` as its first parameter, referring to the specific instance. Python passes `self` implicitly—it is never supplied at the call site. Attributes are created via `self.name`, binding data to the instance.

Listing 1: Defining and using a Student class

```
1 class Student:
2     def __init__(self, name: str, grade: int):
3         self.name = name           # instance attribute
4         self.grade = grade         # instance attribute
5
6     def promote(self) -> str:
7         self.grade += 1
8         return f"{self.name} is now in grade {self.grade}"
9
10    def summary(self) -> str:
11        return f"Student {self.name}, Grade {self.grade}"
12
13 s1 = Student("Alice", 10)
14 s2 = Student("Bob", 11)
15 print(s1.promote())           # Alice is now in grade 11
16 print(s2.summary())           # Student Bob, Grade 11 (unaffected)
```

Notice that modifying `s1.grade` leaves `s2` untouched—each instance holds independent copies of its attributes. **Class attributes**, by contrast, are defined directly in the class body (outside any method) and are shared by all instances. For example, `class Course: institution = "EC University"` creates a single value accessed via `Course.institution` or any instance; modifying it through the class affects all instances. **Instance attributes** declared with `self.attr` inside `__init__` are unique per object.

## 2 Inheritance and Special Methods

## 2.1 Inheritance

Inheritance enables a **subclass** (child) to acquire the attributes and methods of a **superclass** (parent), modelling “is-a” relationships: a `GraduateStudent` *is a* `Student`. The child declares its parent with `class Child(Parent):` and must call `super().__init__(...)` in its own constructor. **Method overriding** redefines a parent method in the child to specialise behaviour—the foundation of **polymorphism**.

Listing 2: Inheritance with constructor delegation and method overriding

```

1 class GraduateStudent(Student):
2     def __init__(self, name: str, grade: int, thesis: str):
3         super().__init__(name, grade)      # delegate to parent
4         self.thesis = thesis
5
6     def summary(self) -> str:               # override parent's summary
7         base = super().summary()           # reuse parent logic
8         return f"{base} -- Thesis: {self.thesis}"
9
10 gs = GraduateStudent("Charlie", 16, "AI Ethics")
11 print(gs.summary())      # Student Charlie, Grade 16 -- Thesis: AI Ethics
12 print(gs.promote())      # Charlie is now in grade 17 (inherited)

```

## 2.2 Special (Dunder) Methods

Python classes can implement **dunder methods** (“double underscore”) to integrate with built-in protocols. Key examples include `__str__` (human-readable, called by `print()`), `__repr__` (unambiguous, called by `repr()`), `__len__` (`len()`), `__eq__` (`==`), `__lt__` (`<`, enables `sorted()`), `__add__` (`+`), and `__getitem__` (indexing).

Listing 3: `__str__`, `__repr__`, and `__len__` for a `Vector2D` class

```

1 class Vector2D:
2     def __init__(self, x: float, y: float):
3         self.x = x; self.y = y
4
5     def __repr__(self): return f"Vector2D({self.x}, {self.y})"
6     def __str__(self):  return f"({self.x}, {self.y})"
7     def __len__(self):  return int((self.x**2 + self.y**2) ** 0.5)
8
9 v = Vector2D(3.0, 4.0)
10 print(v)           # (3.0, 4.0)           -- __str__
11 print(repr(v))     # Vector2D(3.0, 4.0)   -- __repr__
12 print(len(v))      # 5                   -- __len__

```

Listing 4: Implementing `__eq__` and `__lt__` for comparison

```

1 class Book:
2     def __init__(self, title, pages):
3         self.title = title; self.pages = pages
4     def __repr__(self):
5         return f"Book('{self.title}', {self.pages})"
6     def __eq__(self, other):
7         return self.title == other.title and self.pages == other.pages
8     def __lt__(self, other):
9         return self.pages < other.pages
10
11 print(sorted([Book("B",300), Book("A",150), Book("C",450)]))
12 # [Book('A', 150), Book('B', 300), Book('C', 450)]

```

### 3 File Input/Output

Python's `open()` returns a file object for reading, writing, and seeking. Always use the `with` statement—the **context manager protocol**—which guarantees the file is closed when the block exits, even if an exception occurs.

#### 3.1 File Open Modes

The second argument to `open()` is the **mode** string. The default is `'r'` (text read), which raises `FileNotFoundError` if the file does not exist.

| Mode              | Operation        | Behaviour                                                                           |
|-------------------|------------------|-------------------------------------------------------------------------------------|
| <code>'r'</code>  | Read (text)      | Opens existing file; pointer at start.                                              |
| <code>'w'</code>  | Write (text)     | <b>Immediately truncates</b> existing file; creates if absent.                      |
| <code>'a'</code>  | Append (text)    | Pointer at end; creates if absent. Preserves content.                               |
| <code>'x'</code>  | Exclusive create | Creates new file; raises <code>FileExistsError</code> if exists.                    |
| <code>'r+'</code> | Read + Write     | Opens for both; pointer at start. Does not truncate.                                |
| <code>'b'</code>  | Binary suffix    | Combine: <code>'rb'</code> , <code>'wb'</code> , <code>'ab'</code> for binary data. |

**Warning:** Mode `'w'` *immediately destroys* all existing content. Use `'a'` to add data safely.

#### 3.2 Reading and Writing

The file object provides `read()` (entire file), `readline()` (single line), `readlines()` (list of lines), and direct iteration (`for line in f:`, the most memory-efficient approach). Writing uses `write()` and `writelines()`.

Listing 5: Reading and writing with context managers

```

1 # Read entire file
2 with open("data.txt", "r") as f:
3     content = f.read()
4
5 # Iterate line by line --- memory-efficient
6 with open("data.txt", "r") as f:
7     for line in f:
8         print(line.strip())
9
10 # Write --- WARNING: overwrites existing content
11 with open("output.txt", "w") as f:
12     f.write("First line\n")
13     f.write("Second line\n")
14
15 # Append --- safe; preserves existing content
16 with open("log.txt", "a") as f:
17     f.write("New entry\n")

```

### 4 Exception Handling

An **exception** disrupts normal program flow. Python's structured handling—`try`, `except`, `else`, `finally`—enables graceful recovery instead of crashes.

#### 4.1 The `try/except/else/finally` Structure

- **try** — Code that may raise an exception.
- **except** — Catches specific exceptions. Multiple clauses target different types. Never use a bare `except:`.
- **else** — Runs only if no exception occurred (success path).

- **finally** — Runs **always**; use for cleanup (closing resources, restoring state).

Listing 6: Complete try/except/else/finally example

```

1 def safe_divide(a, b):
2     try:
3         result = a / b
4     except ZeroDivisionError:
5         print("Error: Division by zero."); return float("inf")
6     except TypeError:
7         print("Error: Numeric arguments required."); return float("nan")
8     else:
9         print(f"Success: {a} / {b} = {result}"); return result
10    finally:
11        print("safe_divide() finished.\n")
12
13 print(safe_divide(10, 2))    # success path
14 print(safe_divide(5, 0))    # ZeroDivisionError

```

## 4.2 Common Exceptions and Best Practices

| Exception         | Trigger                     | Example             |
|-------------------|-----------------------------|---------------------|
| FileNotFoundError | open() cannot find file     | open("missing.txt") |
| ValueError        | Right type, invalid value   | int("hello")        |
| TypeError         | Incompatible types          | "age: " + 5         |
| ZeroDivisionError | Division or modulo by zero  | 1 / 0               |
| IndexError        | Sequence index out of range | [1,2,3][5]          |
| KeyError          | Dictionary key not found    | { }["missing"]      |

**Best practices:** (1) Catch specific exceptions—a bare `except:` masks `KeyboardInterrupt` and genuine bugs. (2) Keep `try` blocks minimal. (3) Never silently `pass` in an `except` block. (4) Use `finally` for cleanup, `with` for files. (5) Raise exceptions with descriptive messages.

Listing 7: Best practice: specific handling with informative fallback

```

1 import json
2
3 def read_config(filename):
4     try:
5         with open(filename, "r") as f:
6             return json.load(f)
7     except FileNotFoundError:
8         print(f"Config '{filename}' not found. Using defaults.")
9         return {"debug": False}
10    except json.JSONDecodeError as e:
11        print(f"Invalid JSON in '{filename}': {e}. Using defaults.")
12        return {"debug": False}

```

## 5 NumPy: The Foundation of Numerical Python

NumPy is the cornerstone of scientific computing in Python. Its central data structure, the **ndarray** (N-dimensional array), provides a homogeneous, fixed-type container enabling **vectorised operations**—computations applied to entire arrays without explicit Python loops. The entire scientific Python ecosystem (Pandas, Matplotlib, SciPy, Scikit-learn, TensorFlow) builds upon NumPy.

### 5.1 Why NumPy?

A Python list stores references to arbitrary objects scattered in memory; every access requires pointer dereferencing. A NumPy array stores elements of a single fixed type in a **contiguous memory block**, yielding three advantages: (1) operations run in compiled C (10–100× faster

than Python loops); (2) contiguous storage is far more memory-efficient; (3) vectorisation eliminates boilerplate loops—`arr * 2` multiplies every element in one operation.

Listing 8: Speed comparison: list loop versus vectorised array

```
1 import numpy as np, time
2
3 data = list(range(1_000_000))
4 t0 = time.time(); py_sq = [x**2 for x in data]
5 print(f"List: {time.time() - t0:.3f}s")
6
7 arr = np.arange(1_000_000)
8 t0 = time.time(); np_sq = arr ** 2
9 print(f"NumPy: {time.time() - t0:.3f}s")
```

## 5.2 Creating Arrays and Inspecting Properties

NumPy provides factory functions for constructing arrays. The `shape` argument is always a tuple of dimension sizes. Every array carries metadata accessible through attributes: `.shape`, `.ndim`, `.size`, `.dtype`.

Listing 9: Creating arrays and inspecting properties

```
1 import numpy as np
2
3 # From sequences
4 a = np.array([1, 2, 3, 4, 5])           # 1D from list
5 b = np.array([1, 2], [3, 4], [5, 6])    # 2D from nested list
6
7 # Allocators and sequences
8 z = np.zeros((3, 4))                   # 3x4 of 0.0
9 o = np.ones((2, 5))                    # 2x5 of 1.0
10 e = np.eye(4)                           # 4x4 identity
11 r = np.arange(0, 20, 3)                  # [0, 3, 6, 9, 12, 15, 18]
12 l = np.linspace(0, 1, 6)                 # 0.0, 0.2, ..., 1.0
13 rand = np.random.rand(3, 3)              # 3x3 uniform [0,1)
14 ints = np.random.randint(10, 50, (2, 4)) # 2x4 integers in [10, 50)
15
16 # Properties
17 arr = np.ones((3, 4, 2))
18 print(arr.shape, arr.ndim, arr.size, arr.dtype)
19 # (3, 4, 2) 3 24 float64
```

Common dtypes include `int32`, `int64`, `float32`, `float64` (default), and `bool`. Convert with `arr.astype(np.int32)`.

## 6 Indexing, Slicing, and Boolean Masks

NumPy extends Python's indexing with powerful multi-dimensional and boolean-based selection. An essential distinction: list slicing creates a **copy**; NumPy slicing returns a **view** into the same data.

### 6.1 One-Dimensional Slicing

One-dimensional arrays support `[start:stop:step]`. Slices are views—modifying a slice modifies the original.

Listing 10: 1D array slicing

```
1 a = np.array([10, 20, 30, 40, 50, 60, 70])
2 print(a[2:5])      # [30 40 50]           (indices 2-4)
3 print(a[::2])      # [10 30 50 70]        (every second)
4 print(a[::-1])     # [70 60 50 40 30 20 10] (reversed)
5
6 # Slices are VIEWS
```

```

7 view = a[1:4]; view[0] = 999
8 print(a)          # [ 10 999  30  40  50  60  70]

```

## 6.2 Two-Dimensional Indexing

Two-dimensional arrays use **comma-separated** syntax: `[row, column]`. Each axis accepts an integer, slice, or list of indices.

Listing 11: 2D array indexing patterns

```

1 m = np.array([[ 1,  2,  3,  4],
2               [ 5,  6,  7,  8],
3               [ 9, 10, 11, 12]])
4
5 print(m[0, 0])      # 1           (single element)
6 print(m[:, 2])      # [3 7 11]    (entire column 2)
7 print(m[1:, 2:])    # [[ 7,  8],  (bottom-right sub-matrix)
8                   # [11, 12]]
9 print(m[[0, 2], :]) # rows 0 and 2 (fancy indexing)

```

## 6.3 Boolean (Mask) Indexing

Boolean indexing selects elements satisfying a condition. This is one of NumPy's most powerful features for data filtering and conditional assignment.

Listing 12: Boolean indexing and conditional assignment

```

1 scores = np.array([68, 92, 45, 78, 88, 55, 95])
2
3 # Filter and assign
4 above_80 = scores[scores >= 80]          # [92 88 95]
5 middle   = scores[(scores >= 60) & (scores <= 89)] # [68 78 88]
6 scores[scores < 60] = 0                  # replace failing
7 print(scores)                          # [68 92  0 78 88  0 95]
8
9 # np.where for vectorised if-else
10 data = np.array([3, 8, 1, 6, 9, 2])
11 result = np.where(data > 5, data, -1)    # [-1  8 -1  6  9 -1]

```

**Critical:** Combine conditions with `&` (AND) and `|` (OR), *not* Python `and/or`. Each condition must be wrapped in parentheses.

## 7 Vectorised Operations and Broadcasting

### 7.1 Element-Wise Arithmetic and Matrix Multiplication

All standard operators apply **element-wise** to NumPy arrays. For linear algebra matrix multiplication, use the `@` operator (or `np.dot()`). **Do not confuse** `A * B` (element-wise) with `A @ B` (matrix product).

Listing 13: Element-wise operations and matrix multiplication

```

1 a = np.array([1.0, 2.0, 3.0, 4.0])
2 b = np.array([5.0, 6.0, 7.0, 8.0])
3 print(a + b)      # [ 6.  8. 10. 12.] (addition)
4 print(a * b)      # [ 5. 12. 21. 32.] (multiplication)
5 print(a ** 2)     # [ 1.  4.  9. 16.] (power)
6
7 A = np.array([[1, 2], [3, 4]])
8 B = np.array([[5, 6], [7, 8]])
9 print(A * B)      # [[ 5 12]         element-wise (Hadamard)
10                  # [21 32]]
11 print(A @ B)     # [[19 22]         matrix product
12                  # [43 50]]

```

## 7.2 Broadcasting

Broadcasting applies binary operations to arrays of **different shapes** by automatically aligning dimensions:

1. Align shapes from the **rightmost** dimension.
2. Dimensions are compatible if equal, or if one is 1.
3. Missing dimensions are prepended with ones. Size-1 dimensions stretch to match.

| Shape A | Shape B | Result                    |
|---------|---------|---------------------------|
| (3, 4)  | (4,)    | Compatible → (3, 4)       |
| (3, 4)  | (3, 1)  | Compatible → (3, 4)       |
| (3, 4)  | (2, 4)  | Incompatible — ValueError |

Listing 14: Broadcasting examples

```

1 matrix = np.ones((3, 4))           # (3, 4)
2 row_vec = np.array([10, 20, 30, 40]) # (4,) -- broadcast across rows
3 print(matrix + row_vec)             # [[11. 21. 31. 41.] ... ]
4
5 # Centre data: subtract column means via broadcasting
6 data = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
7 centred = data - data.mean(axis=0)   # (3,3) - (3,)
8 print(centred)                       # [[-3. -3. -3.] [ 0.  0.  0.] [ 3.  3.  3.]]

```

## 8 Aggregation and Transformation Functions

NumPy provides efficient mathematical and statistical functions operating on entire arrays or along specified axes. The axis mnemonic: `axis=0` collapses the row dimension (operates column-wise); `axis=1` collapses the column dimension (operates row-wise).

Listing 15: Aggregation with axis control, argmax, and reshape

```

1 data = np.array([[ 1,  3,  5,  7],
2                  [ 2,  4,  6,  8],
3                  [10, 20, 30, 40]])
4
5 print(np.sum(data))                # 136
6 print(np.mean(data, axis=0))        # [4.33  9. 13.67 18.33] (column means)
7 print(np.mean(data, axis=1))        # [ 4.  5. 25.] (row means)
8
9 # Positional functions
10 vals = np.array([15, 42, 8, 23, 91, 16])
11 print(np.argmax(vals))              # 4 (index of 91)
12
13 # Reshaping --- use -1 to auto-infer one dimension
14 arr = np.arange(12).reshape(3, 4)   # 3x4
15 print(arr.reshape(2, -1).shape)     # (2, 6)

```

### 8.1 Practical Worked Example

The following example combines array creation, boolean indexing, broadcasting, and aggregation into a realistic student-score analysis.

Listing 16: Complete NumPy workflow: student score analysis

```

1 scores = np.array([[78, 85, 92, 88],
2                    [65, 70, 72, 68],
3                    [90, 93, 95, 91],
4                    [55, 60, 58, 62],
5                    [82, 88, 85, 90]])
6
7 # Per-student and per-subject averages
8 student_avgs = scores.mean(axis=1)   # [85.75 68.75 92.25 58.75 86.25]

```



```

9
10 # Best student (argmax gives index)
11 best = np.argmax(student_avgs)
12 print(f"Best: index {best}, avg {student_avgs[best]}")
13
14 # Apply curve: +5 points, capped at 100
15 curved = np.where(scores + 5 > 100, 100, scores + 5)
16
17 # Count high scores (>= 90) per student
18 high = np.sum(scores >= 90, axis=1)      # [1 0 4 0 1]

```

## 9 Summary and Key Syntax Patterns

This document has covered the three pillars of Module 2: object-oriented programming, file handling with exception management, and numerical computing with NumPy. The following synopses serve as a quick-reference guide.

### 9.1 Object-Oriented Programming

- **Class:** `class ClassName:` — blueprint. Constructor: `def __init__(self, ...)` initialises `self.attr`.
- **Instantiation:** `obj = ClassName(args)` — `self` passed implicitly.
- **Inheritance:** `class Child(Parent):` — call `super().__init__(...)` first. Override methods to specialise.
- **Dunder methods:** `__str__` (print), `__repr__` (debug), `__len__` (`len()`), `__eq__` (`==`), `__lt__` (`<`).

### 9.2 File Input/Output

- **Open:** with `open(filename, mode)` as `f`: — always.
- **Modes:** `'r'` (read), `'w'` (write/truncate), `'a'` (append), `'b'` (binary).
- **Read:** `f.read()`, `f.readline()`, or for `line in f`: (memory-efficient).
- **Write:** `f.write(string)`, `f.writelines(lines)` (add `\n` manually).

### 9.3 Exception Handling

- **Structure:** `try/except/else/finally`. Catch specific exceptions; never `except:`. Never pass silently.
- **Common:** `ValueError`, `TypeError`, `ZeroDivisionError`, `IndexError`, `KeyError`, `FileNotFoundError`.

### 9.4 NumPy Arrays

- **Import:** `import numpy as np`. **Create:** `np.array()`, `np.zeros()`, `np.ones()`, `np.eye()`, `np.arange()`, `np.linspace()`, `np.random.rand()`.
- **Properties:** `.shape`, `.ndim`, `.size`, `.dtype`. **Reshape:** `.reshape()`, `-1` for auto-infer.
- **Slicing:** 1D `[start:stop:step]`; 2D `[row, col]`. Slices are **views**, not copies.
- **Boolean indexing:** `arr[arr > threshold]`. Use `&|` with parentheses.
- **Element-wise:** `+`, `-`, `*`, `/`, `**`. **Matrix multiply:** `A @ B` (not `A * B`).
- **Broadcasting:** Align shapes from right; size-1 stretches. Enables `(3,4) + (4,)`.
- **Aggregation:** `np.sum()`, `np.mean()`, `np.max()`, `np.argmax()`; use `axis=0` or `1`.
- **Utility:** `np.where(cond, x, y)` for vectorised if-else.

---

This reference accompanies the Module 2 Jupyter notebook.

For hands-on exercises, open `Module_2_OOP_Files_Numpy.ipynb`.

*Reading is the map; writing code is the journey. Practise extensively.*